

```

top - 08:06:18 up 1 min, 1 user, load average: 0.16, 0.09, 0.03
tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.7 sy, 0.0 ni, 99.3 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7754.6 total, 6379.1 free, 543.0 used, 1083.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7211.7 avail Mem

  PID USER      DS  NI      VIRT    RES    SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0  21892  13296  9720  S   0.0   0.2   0:01.62 systemd
    2 root        20   0   3100   2204  2972  S   0.0   0.0   0:00.62 init-systemd(0b
    6 root        20   0   3196   2128  2088  S   0.0   0.0   0:00.01 init
   38 root       19  -1  42168  16172  15108  S   0.0   0.2   0:00.35 systemd-journal
  160 root        20   0  25280   6584   5124  S   0.0   0.1   0:00.30 systemd-udev
  323 systemd+    20   0  21304  13184  11856  S   0.0   0.2   0:00.22 systemd-resolve
  329 systemd+    20   0  91836   7936   6972  S   0.0   0.1   0:00.15 systemd-timesyn
  387 root        20   0   4244   2604  2032  S   0.0   0.0   0:00.01 cron
  388 message+    20   0   9544   5356   4656  S   0.0   0.1   0:00.13 dbus-daemon
  390 root        20   0  17980   8848   7812  S   0.0   0.1   0:00.20 systemd-logind
  396 root        20   0  175628  13064  11040  S   0.0   0.2   0:00.20 wsl-pre-service
  418 root        20   0   3124   1968   1832  S   0.0   0.0   0:00.00 Agetty
  420 syslog      20   0  222516   8084  4588  S   0.0   0.1   0:00.15 rsyslogd
  427 root        20   0  107036  23144  13604  S   0.0   0.3   0:00.32 unattended-upgr
  437 root        20   0   3180   1180   908  S   0.0   0.0   0:00.00 SessionLeader
  440 root        20   0   3200   1248   1104  S   0.0   0.0   0:00.02 Relay(329)
  448 teja       20   0   6876   5300  3660  S   0.0   0.1   0:00.07 bash
  450 root        20   0   6696   4652  2876  S   0.0   0.1   0:00.03 login
  479 teja       20   0  20332  11488   9356  S   0.0   0.1   0:00.20 systemd
  480 teja       20   0  21168  3572   1848  S   0.0   0.0   0:00.00 (sd-pam)
  484 teja       20   0   6080   5360  3700  S   0.0   0.1   0:00.05 bash
  482 root        20   0  25288   3720  2228  S   0.0   0.0   0:00.00 (udev-worker)
  483 root        20   0  25288   3720  2228  S   0.0   0.0   0:00.00 (udev-worker)
  494 teja       20   0   9300   5416  3236  R   0.0   0.1   0:00.01 top

```

```

teja@LAPTOP-FGDPB110:~$ sleep 100 &
[1] 459
teja@LAPTOP-FGDPB110:~$ sleep 200 &
[2] 460
teja@LAPTOP-FGDPB110:~$ fg %1
sleep 100
bg %1
^C
teja@LAPTOP-FGDPB110:~$ bg %1
-bash: bg: %1: no such job
teja@LAPTOP-FGDPB110:~$ kill %1
-bash: kill: %1: no such job
teja@LAPTOP-FGDPB110:~$ kill %1
-bash: kill: %1: no such job
teja@LAPTOP-FGDPB110:~$ kill %2
teja@LAPTOP-FGDPB110:~$ pgrep sleep
[2]+  Terminated                  sleep 200
teja@LAPTOP-FGDPB110:~$ pgrep sleep
teja@LAPTOP-FGDPB110:~$ renice 5 -p PID
renice: bad process ID value: PID
teja@LAPTOP-FGDPB110:~$ ps -o pid,comm,pri,ni -p PID
error: process ID list syntax error

```

Usage:

ps [options]

Try 'ps --help <simple|list|output|threads|misc|all>'
 or 'ps --help <s|l|o|t|m|a>'
 for additional help text.

For more details see ps(1).

```
teja@LAPTOP-FGDPB110:~$ |
```

```

C:\Program Files\Git\bin> cd C:\Program Files\Git\bin
context.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();
    if (pid == 0)
    {
        for (int i = 1; i <= 5; i++)
        {
            printf("Child process: %d\n", i);
            sleep(1);
        }
    }
    else
    {
        for (int i = 1; i <= 5; i++)
        {
            printf("Parent process: %d\n", i);
            sleep(1);
        }
    }
    return 0;
}

```

```

For more details see ps(1).
teja@LAPTOP-FGDPB110:~$ nano context.c
teja@LAPTOP-FGDPB110:~$ gcc context.c -o context
teja@LAPTOP-FGDPB110:~$ ./context
Parent process: 1
Child process: 1
Child process: 2
Parent process: 2
Child process: 3
Parent process: 3
Parent process: 4
Child process: 4
Parent process: 5
Child process: 5
teja@LAPTOP-FGDPB110:~$ |

```

Short theory

The parent and child processes execute alternately. The operating system saves one process state and loads another process state. This operation is called context switching.

```

teja@LAPTOP-FGDPB110:~$ vmstat 1 5
procs -----memory----- --swap-- --io-- --system-- -----cpu-----
r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st gu
1  0       0 7356360  5212 196640    0    0   800   17  261    0  0  0 100  0  0  0
0  0       0 7356360  5212 196640    0    0    0    0  176   68  0  0 100  0  0  0
0  0       0 7356360  5212 196712    0    0    0    0  159   64  0  0 100  0  0  0
0  0       0 7356360  5212 196712    0    0    0    0   73   36  0  0 100  0  0  0
0  0       0 7356360  5212 196712    0    0    0    0   77   36  0  0 100  0  0  0
teja@LAPTOP-FGDPB110:~$ chrt -p $$
pid 329's current scheduling policy: SCHED_OTHER
pid 329's current scheduling priority: 0
teja@LAPTOP-FGDPB110:~$ |

```

In this session, we studied preemptive scheduling and context switching. Round Robin gives each process a fixed time quantum. Priority scheduling selects the process with the highest priority. We also studied race conditions, critical sections, deadlock, starvation and aging. A deadlock occurs when processes wait permanently for resources held by one another. Aging prevents starvation by gradually increasing the priority of a waiting process.